

数据算法理论说明文档 (Data Algorithm Theory Documentation)

本文档详细阐述 DevScope 项目中的核心算法理论，包括输入变量的数学定义、模型参数设定以及具体的计算路径。所有数学公式均采用 LaTeX 格式书写，以确保严谨性。

1. OpenRank 计算原理与系统实现 (Core Requirement)

OpenRank 是面向开源生态的影响力评分，基于协作网络的传播模型（类似 PageRank）。每个节点的得分会从其入边累积并按出边分摊，平衡由阻尼因子控制：

$$\text{extOpenRank}(v) = \alpha \sum_{u \in \text{in}(v)} \frac{\text{OpenRank}(u)}{\text{out}(u)} + (1 - \alpha) \cdot \text{base}$$

- 节点：开发者、仓库、Issue、PR 等。
- 边权：Commit、Review、Issue/PR 互动等贡献行为的加权。
- 阻尼因子 α ：常取 0.85，用于防止循环放大并引入基础分布。
- 时间衰减：近期行为权重更高（由官方离线计算体现）。

本项目对 OpenRank 的处理遵循如下“严格与代码一致”的规则：

- 概念层面：OpenRank 衡量的是开发者或仓库在开源网络中的综合影响力，来源于官方离线计算结果；本项目不在本地重算该指标，上式用于原理说明。
- 数据来源：通过 OpenDigger 官方静态 API 获取 JSON 数据。
 - 开发者： `https://oss.x-lab.info/open_digger/github/{username}/openrank.json`
 - 仓库： `https://oss.x-lab.info/open_digger/github/{owner}/{repo}/openrank.json`
- 年度值选择：仅从键形如 "YYYY" 的年度条目中选取最新年份的值进行展示；形如 "YYYY-MM" 的月度条目不参与选择。
- 数值处理：返回值统一保留两位小数；若 API 返回 404 或解析失败则视为无数据，返回 None（前端显示为 N/A）。
- 代码对应：
 - 开发者 OpenRank： `backend/opendigger_client.py` 中的 `get_user_openrank(username, timeout)` 完成上述逻辑（包含年度筛选与两位小数保留）。
 - 仓库 OpenRank： `backend/opendigger_client.py` 中的 `get_repo_openrank(owner, repo, timeout)` 完成相同的年度筛选与返回处理。

一致性声明：本文档的 OpenRank 描述与当前实现完全一致。若未来需要在本地重算 OpenRank，则需另行引入关系图构建、时间衰减权重、迭代求解等模块，不属于本项目当前范围。

2. 符号定义与参数表 (Symbol Definitions and Parameters)

1.1 变量符号定义 (Variable Symbols)

符号 (Symbol)	定义 (Definition)	数据来源/计算方式 (Source/Calculation)
U	目标开发者 (User)	用户输入的 GitHub ID
T_{now}	当前时间 (Current Timestamp)	系统当前 UTC 时间
W_{obs}	观测窗口 (Observation Window)	$[T_{now} - 365days, T_{now}]$
$\mathcal{C}$	提交时间戳集合 (Commit Timestamps)	<code>github_client.get_user_commit_activity</code>
t_i	第 i 次提交的时间点	$t_i \in \mathcal{C}, t_i \in W_{obs}$
Δt_i	相邻提交的时间间隔 (Time Interval)	$\Delta t_i = t_{i+1} - t_i$
$\mathcal{R}$	用户参与的仓库集合 (Repositories)	<code>github_client.get_user_repos_union</code> (拥有仓库 $\cup$ 参与贡献仓库)
$\mathcal{L}$	技术标签/语言集合 (Topics/Languages)	从 $\mathcal{R}$ 中提取的 <code>language</code> 和 <code>topics</code> ; Topic 仅指真实仓库 <code>topics</code> 标签，不对语言兜底
N_{proj}	参与项目总数 (Total Projects)	\$
n_k	特定技术 k 的出现频次	Count of topic k in $\mathcal{L}$
K	技术类别的总数 (Total Categories)	\$
M_{od}	OpenDigger 宏观指标集合	<code>opendigger_client.get_developer_metrics</code>

符号 (Symbol)	定义 (Definition)	数据来源/计算方式 (Source/Calculation)
$P(T_k)$	开发者对技术 k 的倾向概率	建模计算结果
$P(Active)$	未来 30 天活跃概率	建模计算结果
S_{match}	综合匹配度得分	建模计算结果

1.2 模型参数定义 (Model Parameters)

符号 (Symbol)	参数名称 (Parameter Name)	设定值 (Value)	说明 (Description)
α	拉普拉斯平滑系数 (Laplace Smoothing Factor)	1.0	防止零概率问题
θ_{cold}	冷启动阈值 (Cold Start Threshold)	5	项目数少于此值触发冷启动逻辑
θ_{conf}	置信度全量阈值 (Confidence Threshold)	10	项目数达到此值时置信度为 1.0
w	置信度权重 (Confidence Weight)	$[0, 1]$	动态计算，取决于 N_{proj}
k	Weibull 分布形状参数 (Shape Parameter)	拟合得出	描述活跃率随时间的变化趋势
λ	Weibull 分布尺度参数 (Scale Parameter)	拟合得出	描述平均活跃间隔的特征尺度
β_{tech}	匹配分-技术权重 (Tech Weight)	0.7	技术契合度在总分中的占比
β_{active}	匹配分-活跃权重 (Active Weight)	0.3	活跃度在总分中的占比

3. 输入变量详解 (Input Variables Detail)

本系统的输入变量 X 由两部分组成： $X = \{D_{github}, D_{od}\}$ 。

2.1 GitHub 实时行为数据 (D_{github})

通过 `github_client.py` 获取，作为微观行为建模的基础。

1. 提交时间序列 ($\mathcal{C}$)

- 来源: `get_user_commit_activity(username)`
- 定义: $\mathcal{C} = \{t_1, t_2, \dots, t_m\}$
- 约束:
 - $t_1 < t_2 < \dots < t_m$ (升序排列)
 - $\forall t \in \mathcal{C}, t \geq T_{now} - 365\text{days}$ (严格限制在最近一年窗口内)
- 用途: 用于活跃时间分布拟合 ($P(Active)$)。

2. 仓库与技术特征 ($\mathcal{R}, \mathcal{L}$)

- 来源: `get_repos(username)`
- 定义: $\mathcal{R} = \{r_1, r_2, \dots, r_n\}$
- 元素结构: 每个 r_j 包含 `{name, language, topics, stargazers_count}`。
- 特征提取: $\mathcal{L} = \bigcup_{r \in \mathcal{R}} (\{r.language\} \cup r.topics)$
- 用途: 用于技术倾向性预测 ($P(T_k)$)。

3. 用户基础画像 ($U_{profile}$)

- 来源: `get_user(username)`
- 包含: `{login, public_repos, followers, following}`
- 用途: 提供 N_{proj} 用于冷启动判断。

2.2 OpenDigger 宏观指标数据 (D_{od})

通过 `opendigger_client.py` 获取，作为宏观能力参考。

1. 开发者指标集合 (M_{od})

- 来源: `get_developer_metrics(username, data)`
- 定义: $M_{od} = \{m_{rank}, m_{activity}, m_{stars}, \dots\}$
- 具体指标:
 - m_{rank} : OpenRank 值，衡量在开源网络中的影响力。
 - $m_{activity}$: 长期活跃度指数。
- 用途: 辅助展示，不直接参与概率分布拟合，但在冷启动或数据缺失时可作为定性参考。

4. 计算路径与数学模型 (Calculation Path and Mathematical Models)

4.1 活跃时间分布建模 (Activity Time Distribution Modeling)

目标是预测开发者下一次活跃的时间概率。我们假设开发者的活跃间隔服从 **Weibull 分布**。

4.1.1 间隔序列生成

首先计算相邻提交的时间间隔序列 ΔT :

$$\Delta T = \{\Delta t_i \mid \Delta t_i = t_{i+1} - t_i, \quad 1 \leq i < m\}$$

单位通常转换为 **天 (Days)**。

4.1.2 Weibull 分布拟合 (MLE)

假设概率密度函数 (PDF) 为:

$$f(t; k, \lambda) = \frac{k}{\lambda} \left(\frac{t}{\lambda}\right)^{k-1} e^{-(t/\lambda)^k}, \quad t \geq 0$$

其中:

- $k > 0$ 是形状参数 (Shape Parameter)。
- $\lambda > 0$ 是尺度参数 (Scale Parameter)。

使用 **最大似然估计 (Maximum Likelihood Estimation, MLE)** 求解参数 $\hat{k}, \hat{\lambda}$:

$$(\hat{k}, \hat{\lambda}) = \underset{k, \lambda}{\operatorname{argmax}} \sum_{i=1}^{|\Delta T|} \ln f(\Delta t_i; k, \lambda)$$

注: 工程实现中使用 `scipy.stats.weibull_min.fit`。

4.1.3 备选模型: 指数分布 (Exponential Distribution)

若 Weibull 拟合失败或数据稀疏, 退化为指数分布 (无记忆性假设):

$$f(t; \lambda) = \lambda e^{-\lambda t}$$

4.1.4 活跃概率预测 (Prediction)

计算在未来 τ 天内（例如 $\tau = 30$ ）再次活跃的累积概率 (CDF):

$$P(\text{Active} \leq \tau) = F(\tau) = \int_0^\tau f(t) dt = 1 - e^{-(\tau/\hat{\lambda})^{\hat{k}}}$$

同时计算期望等待时间 (Mean Recurrence Time):

$$E[T] = \hat{\lambda} \cdot \Gamma(1 + 1/\hat{k})$$

其中 $\Gamma(\cdot)$ 为 Gamma 函数。

4.2 技术倾向性建模 (Technical Tendency Modeling)

目标是估计开发者在特定技术领域 T_k 的投入概率 $P(T_k)$ 。

4.2.1 话题权重构造 (Topic Weighting)

为避免当每个话题仅在一个仓库中出现时概率过于均匀的状况，我们在统计 n_k 前为仓库级话题引入“影响力权重”，结合仓库星标与用户在该仓库的提交次数：

$$w_r = \max(1, \lfloor \log(1 + \text{stars}_r) \rfloor + \lfloor \log(1 + \text{commits}_r) \rfloor)$$

其中：

- stars_r : 仓库 r 的 `stargazers_count`。
- commits_r : 目标用户在仓库 r 于 W_{obs} 窗口中的提交条数，来源于 `github_client.get_user_commit_activity`。

据此，针对话题 T_k 的加权计数为：

$$n_k = \sum_{r \in \mathcal{R}} (1[T_k \in r.\text{topics}] \cdot w_r)$$

该设计使得热门仓库与用户高投入仓库对对应话题的贡献更大，从而拉开概率差异。

4.2.2 贝叶斯平滑估计 (Bayesian Smoothing)

在得到加权计数 n_k 后，基于多项分布假设，使用拉普拉斯平滑 (Laplace Smoothing) 计算后验概率，以解决小样本下的零概率问题：

$$P_{user}(T_k) = \frac{n_k + \alpha}{N_{total} + \alpha \cdot K}$$

其中：

- n_k : 技术 k 在 $\mathcal{L}$ 中出现的次数。
- $N_{total} = \sum n_k$: 总技术标签数。
- K : 唯一技术标签的总数。
- $\alpha = 1$: 平滑系数。

4.2.3 冷启动加权融合 (Cold Start Weighted Fusion)

当用户项目数 N_{proj} 较少时，用户自身的统计数据具有高方差。此时引入社区先验分布 $P_{community}$ 进行融合。

步骤 1: 计算置信度权重 w

$$w = \min \left(1.0, \frac{N_{proj}}{\theta_{conf}} \right)$$

若 $N_{proj} < \theta_{cold}$ ，则 w 显著小于 1，表明主要依赖社区数据。

步骤 2: 概率融合

最终概率 $P_{final}(T_k)$ 为用户分布与社区分布的线性组合：

$$P_{final}(T_k) = w \cdot P_{user}(T_k) + (1 - w) \cdot P_{community}(T_k)$$

其中 $P_{community}$ 根据开发者的推测类型（如 "Backend", "AI/ML"）选取预置的先验分布。

备注：若目标用户近一年内不存在任何真实话题（未设置 `topics`），则前端仅展示语言分布，并在“技术倾向预测”区块提示“暂无Topic倾向数据”。引力图亦回退为语言关系图。

4.3 综合匹配度打分 (Comprehensive Match Scoring)

目标是量化开发者与特定技术栈 $Target$ 的契合程度。

3.3.1 评分公式

$$S_{match} = \beta_{tech} \cdot P_{final}(Target) + \beta_{active} \cdot P(Active \leq 30)$$

3.3.2 解释

- **技术契合度** ($\beta_{tech} \cdot P_{final}$): 反映开发者在历史上使用该技术的频率及倾向。
- **活跃度贡献** ($\beta_{active} \cdot P(Active)$): 反映开发者近期的活跃状态。即使技术栈完全匹配，如果开发者已停止活跃，总分也会受到惩罚。

3.3.3 评分等级映射

Level =

极高匹配 (Excellent)

高度匹配 (High)

中等匹配 (Medium)

低度契合 (Low)

不匹配 (Mismatch)

$S_{match} \geq 0.8$

$0.6 \leq S_{match} < 0.8$

$0.4 \leq S_{match} < 0.6$

$0.2 \leq S_{match} < 0.4$

$S_{match} < 0.2$